

Database Introduction

COSC 304 – Introduction to Database Systems





What is a database?

A **database** is a collection of logically related data for a particular domain.

A **database management system (DBMS)** is software used to define, store, query, update, and manage databases.

- e.g. MySQL, PostgreSQL, SQL Server, Oracle, SQLite, MongoDB

Bottom line: A **database** is the **data** stored, and a **database management system** is the **software** that manages the data.

Databases in the Real-World

Databases support virtually every major digital service: retail, banking, healthcare, transportation, universities, social media, games, and AI applications.

Think about the applications you have used today. What data are they storing about you or for you?

Developing without a Database

Without a database, applications use files to store data *persistently*. A **file-based system** has several problems: code and data duplication, high maintenance costs, and difficulty in supporting multiple users.

- There is no **program-data independence** separating the application from the data it is manipulating. If the data file changes, the code must be changed.

Example: You have been asked to build a simple inventory/sales program for a store. What challenges would you face without a DBMS?

Data Independence and Abstraction

Databases provide *data abstraction* allowing the internal representation of the data to change without affecting programs that use the object through an external definition.

- The DBMS takes the description of the data and handles the low-level details of how to store it, retrieve it, and provide concurrent access to it.

Database Management System Properties

A database management system provides *efficient*, *convenient*, and *secure multi-user* storage and access to *persistent* data.

Persistent - Data remains stored after the application terminates and across system restarts.

Efficient - Supports efficient retrieval and modification of data.

Convenient - Easy to write queries to retrieve data.

Multi-user - More than one user can access and update data at the same time while preserving consistency.

Secure - controls who can access and modify data.

Scalable - can support increasing amounts of data and workload.



Database Terminology

A **data model** is a collection of concepts used to describe the structure of a database. e.g. relational, document, key-value, graph, object

- In the relational model, data is represented using relations (tables), attributes (columns), and tuples (rows).

Data Definition Language (DDL) allows the user to create data structures in the data model used by the database. A **schema** is a description of the structure of the database and is maintained and stored in the **system catalog**. The schema is **metadata**.

- A schema contains structures, names, and types of data stored.

Once a database has been created using DDL, the user accesses data using a **Data Manipulation Language (DML)**.

- The DML allows for the insertion, modification, retrieval, and deletion of data.

SQL is the standard language for relational databases. It includes DDL for defining database structures and DML for manipulating and querying data.

SQL DDL and DML Examples

Create a table to store data on products:

```
CREATE TABLE product (  
    sku            VARCHAR(10),  
    name           VARCHAR(40),  
    description    VARCHAR(50),  
    inventory      INTEGER,  
    PRIMARY KEY (sku));
```

Insert product into database:

```
INSERT INTO product VALUES ('1234', 'Soap', 'Ivory', 100);
```

Retrieve all products where inventory < 10:

```
SELECT name, inventory FROM product WHERE inventory < 10;
```

Database Properties Question

Question: True or False: The data in a database is lost when the power to the computer is turned off.

A) true

B) false

Database Abstraction Question

Question: Defining a database schema using DDL is most similar to defining what in object-oriented programming?

A) Objects

B) Classes

C) Inheritance

D) Polymorphism

DDL vs. DML Question

Question: If you are querying data in a database, which language are you using:

A) DML

B) DDL

C) schemas

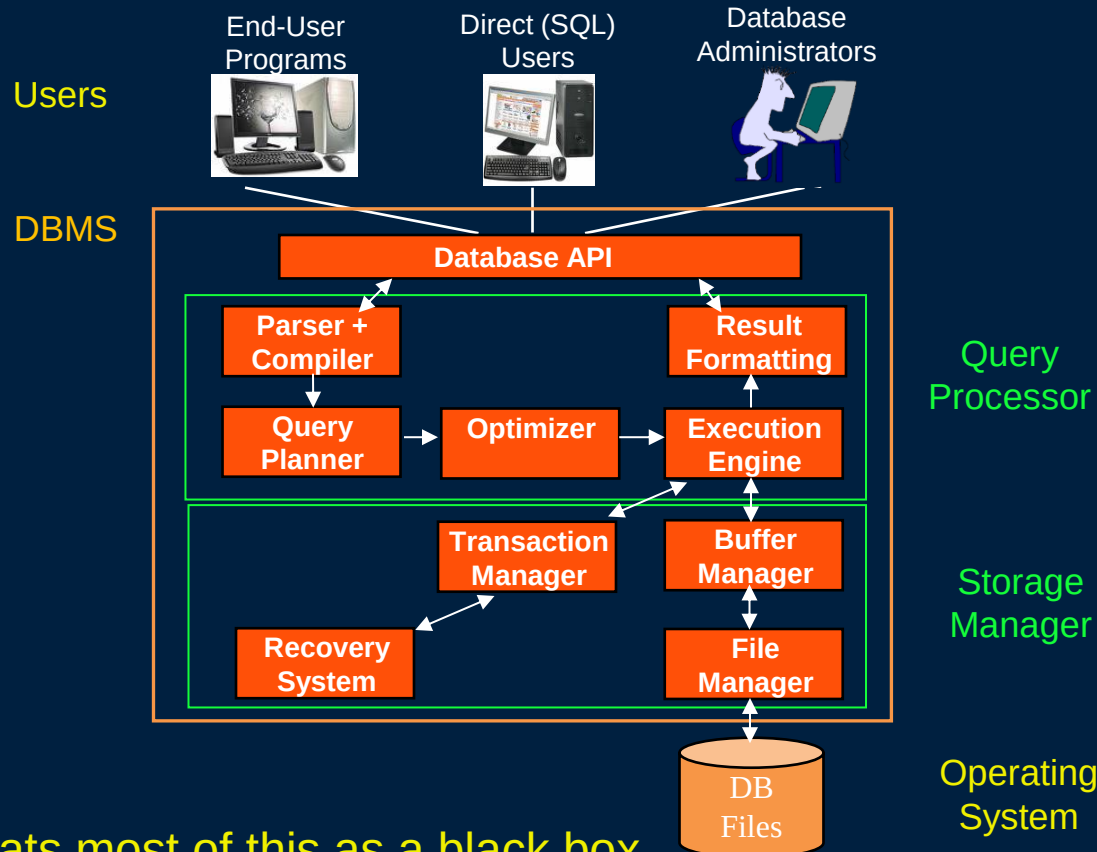
D) Java

Components of a DBMS

A DBMS is a complicated software system containing many components:

- **Query processor** - translates user/application queries into low-level data manipulation actions.
 - Sub-components: query parser, query optimizer
- **Storage manager** - maintains storage information including memory allocation, buffer management, and file storage.
 - Sub-components: buffer manager, file manager
- **Transaction manager** - performs scheduling of operations and implements concurrency control algorithms.

DBMS Architecture



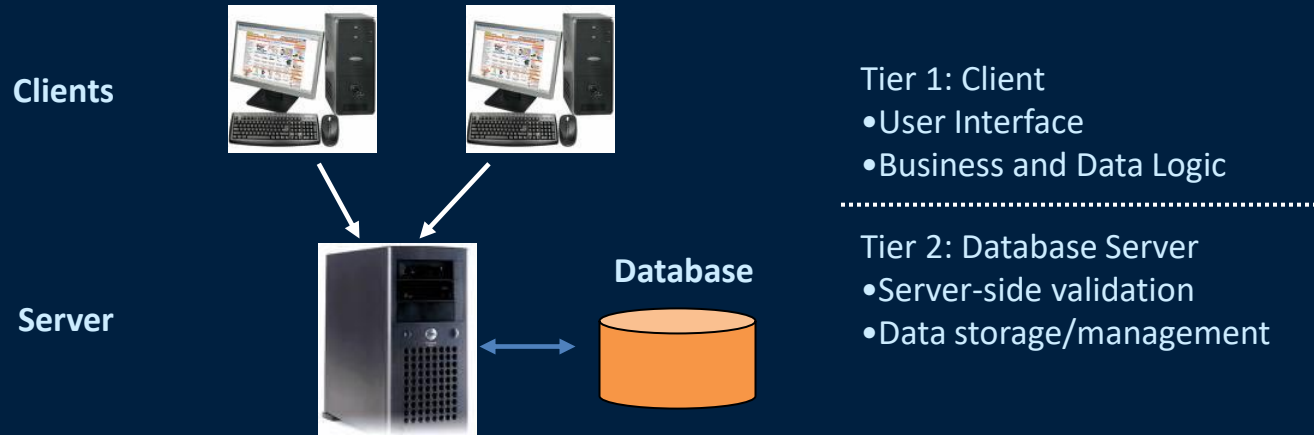
COSC 304 treats most of this as a black box.

Database Architectures

There are several common database application architectures:

- **Embedded/local architecture** - The DBMS runs as part of the application and accesses a local database file (e.g., SQLite or Microsoft Access).
- **Two-Tier client-server architecture** - Client applications connect directly to a separate database server that runs the DBMS (e.g., MySQL, PostgreSQL, SQL Server).
- **Three-Tier application-server architecture** - Clients connect to an application server that contains the business logic and accesses the database server on their behalf (e.g., web browser → web/API server → database server).

Two-Tier Client-Server Architecture



Advantages:

- Centralized storage and administration and one DBMS manages concurrency/recovery
- Multiple applications can share the database
- Simple application architecture

Limitations

- Clients require database connectivity and credentials
- Business/data-access logic may be duplicated across applications
- Exposing the database directly to many clients can complicate security and scaling

Three-Tier Application-Server Architecture



Tier 1: Client (Web/mobile)

- User Interface



Tier 2: Application Server

- Business logic
- Data processing logic



Database



Tier 3: Database Server

- Data validation
- Data storage/management

Advantages:

- Database credentials stay on the application server
- Centralizes business logic
- Easier to change clients independently
- Enables APIs, connection pooling, caching and load balancing
- Better suited to web/mobile applications

Database People

There are several different types of database personnel:

- **Database administrator (DBA) / database platform engineer** - deploys, secures, monitors and maintains DBMSs.
- **Data administrator / data governance manager** - organizational responsibility for what data means, who owns it, how it may be used, retention/security policies, standards, quality, and governance.
- **Database designer/architect** - designs schemas and organization-wide data models.
- **Data engineer** – builds systems and pipelines for collecting, transforming, and storing data.
- **Application developer** – writes software that uses databases.
- **Data analyst/data scientist** – queries and analyzes stored data.
- **DBMS developer** - develops the database system software itself.
- **User** - uses the database directly or through applications.

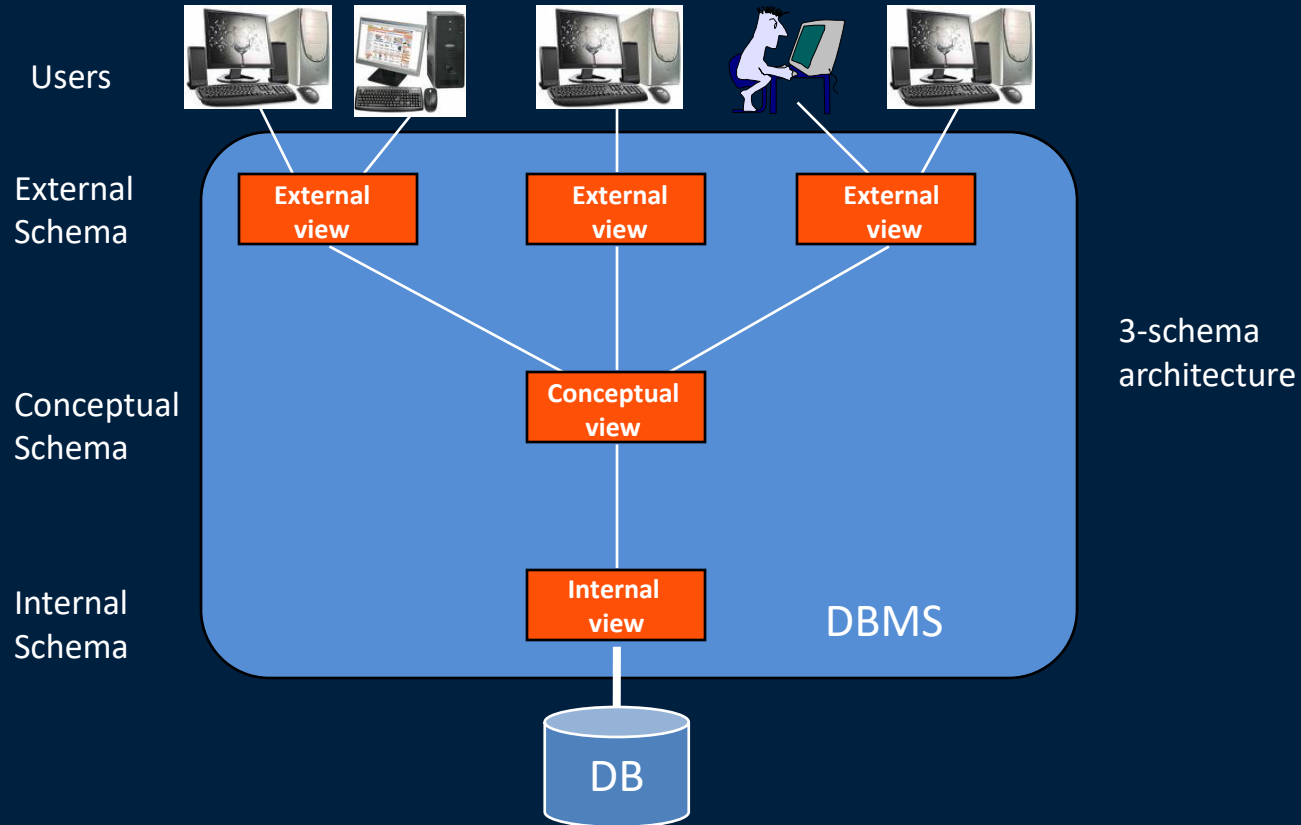
ANSI/SPARC Architecture

One of the major advantages of database systems is data abstraction. Data abstraction is achieved by defining different views of the data. Each view isolates higher-level views from data representation details.

The ANSI/SPARC architecture consists of three levels:

- **Internal Level** - The physical representation of the database on the computer.
How the data is stored.
- **Conceptual Level** - The logical structure of the database that describes *what* data is stored and its relationships.
- **External Level** - The *user's view* of the database that provides the part of the database relevant to the user.

ANSI/SPARC Architecture



Benefits of Three Schema Architecture

External level – Each user/application can have its own view of the database.

Conceptual level – Provides a single shared logical representation of the database for all applications and users independent of physical data storage.

Logical data independence: changing the conceptual schema while preserving users' external views.

- Example: reorganize underlying tables while maintaining a compatible view for an application.

Internal level – Defines how the database is physically stored.

Physical data independence: physical storage can change without changing the conceptual schema.

- Example: add an index or move data to different storage without changing application queries.

Microsoft Access and the Three Schema Architecture

External Level:

- Microsoft Access does not call them views, but you can store queries and use the results in other queries (like a view).
 - External schema is the query (view) name and the attribute metadata.

Conceptual Level:

- All tables and field definitions are in the schema (accessible from the Tables tab).
 - Note that conceptual **schema** is not the data but the metadata.

Internal Level:

- Access represents all data in a single file whose layout it controls.
- The system processes this raw data file by knowing locations and offsets of relations and fields.

ANSI/SPARC Architecture - Three Levels of Views

Question: What are the three levels of views in the ANSI/SPARC architecture starting with the view closest to the user?

- A)** Internal, Conceptual, External
- B)** External, Internal, Conceptual
- C)** Internal, External, Conceptual
- D)** External, Conceptual, Internal
- E)** User, Logical, System

ANSI/SPARC Architecture - Abstraction with Views

Question: Assume you have a Java program accessing a database. Select **one** true statement.

- A)** The file organization is changed. The internal view is where this change is made.
- B)** A field is added to the database. The conceptual view is changed.
- C)** A user should see only a subset of the database. The external view is where this is defined.
- D)** More than one of the above

Conclusion

A **database** is a collection of logically related data managed by a **database management system** (DBMS).

A DBMS provides data independence and abstraction and standard solutions for storage, querying, concurrency, security, integrity, and recovery.

DDL defines database structures. DML queries and manipulates database data.

The three-schema architecture separates external, conceptual, and internal representations of data.

Objectives

- Define: database, DBMS, schema, metadata
- Describe the features of a file-based system and some limitations inherent in that architecture.
- Explain data abstraction and data independence and why they are important.
- Distinguish DDL and DML and identify examples of each.
- Describe at a high level the major services provided by a DBMS.
- Describe several common roles involved in database systems and data management.
- Explain how a schema differs from data.
- Explain the external, conceptual, and internal levels of the three-schema architecture.
- Distinguish logical and physical data independence.
- Compare embedded/local, two-tier client-server, and three-tier application-server architectures.



THE UNIVERSITY OF BRITISH COLUMBIA

